



Today's agenda

↳ Two sum

↳ Container with most water

↳ Sort 0,1,2 → dutch national flag



AlgoPrep



Q) Two Sum

↳ Given N distinct sorted ele, check if there exists a pair (i, j) such that $A[i] + A[j] == K$ $[i \neq j]$

Ex: $A[] \rightarrow \{3, 7, 8, 12, 19\}$, $K=15 \rightarrow \text{true}$

$A[] \rightarrow \{2, 5, 8, 11, 15\}$, $K=16 \rightarrow \text{true}$

$A[] \rightarrow \{1, 3, 9, 20, 27\}$, $K=14 \rightarrow \text{false}$

Idea 1

↳ check all pairs.

T.C: $O(N^2)$

S.C: $O(1)$

Idea 2

↳ using Hashmap

T.C: $O(N)$

S.C: $O(N)$

$$A[i] + A[j] == K$$

$$A[i] + A[j] > K \rightarrow j--$$

$$A[i] + A[j] < K \rightarrow i++$$



Idea 3

$$K = 17$$

$A[] = \{-3, 0, 1, 3, 6, 8, 11, 14, 18, 25\}$

$A[i]$	$A[j]$	Sum	*
-3	25	22	$j--$
-3	18	15	$i++$
0	18	18	$j--$
0	14	14	$i++$
1	14	15	$i++$
3	14	17	$\rightarrow$ return true

until $i < j$

$O(N^2)$ Pairs $\rightarrow O(N)$ Pairs

100 Pairs

10 Pairs

$A[] = \{ -3, 0, 1, 3, 6, 8, 11, 14, 18, 25 \}$
 $K = 17$

$0, 9 ??$

$(0, 8) ??$

~~$1, 9$~~

~~$(0, 7)$~~

$A[0] + A[8] = 15 < K$

~~$2, 9$~~

~~$(0, 6)$~~

$\downarrow A[0] + A[7] = 14 < K$

~~$3, 9$~~

~~$(0, 5)$~~

~~$4, 9$~~

~~$(0, 4)$~~

~~$5, 9$~~

~~$(0, 3)$~~

~~$6, 9$~~

~~$(0, 2)$~~

~~$7, 9$~~

~~$(0, 1)$~~

~~$8, 9$~~

$A[0] + A[9] = 22 > K \rightarrow j--$

$\uparrow A[1] + A[9] = 22 > K$

$\uparrow A[2] + A[9] = 22 > K$

.

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|



// Pseudo code

```
boolean check (int arr[N], int K) {  
    int i = 0;  
    int j = N-1;  
  
    while (i < j) {  
        if (arr[i] + arr[j] == K) { return true; }  
        else if (arr[i] + arr[j] > K) { j--; }  
        else { i++; }  
    }  
    return false;  
}
```

T.C: $O(N)$
S.C: $O(1)$

$arr[i] + arr[j] == K$

$arr[i] + arr[j] > K \rightarrow j--$

$arr[i] + arr[j] < K \rightarrow i++$



* Things to focus while applying 2 Pointers.

↳ if you are checking all the Pairs, try optimizing it with two Pointers.

1) where to initialize two pointers.

2) update your pointers.

3) when to stop.

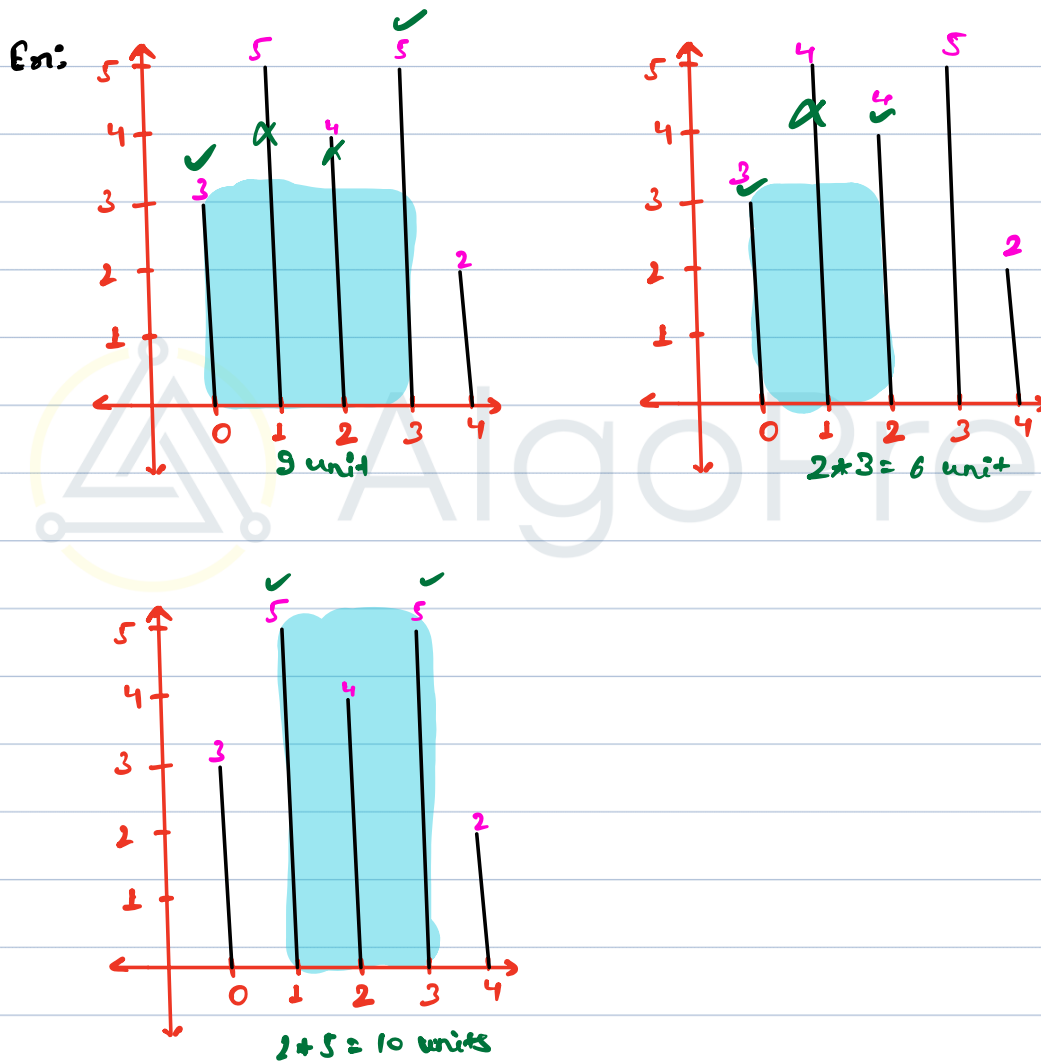


AlgoPrep



Q) Container with most water

↳ Given an $arr[N]$, where $arr[i]$ represents height of each wall. Pick any 2 walls s.t max water is accumulated between them.



//ideal

↳ check all Pairs

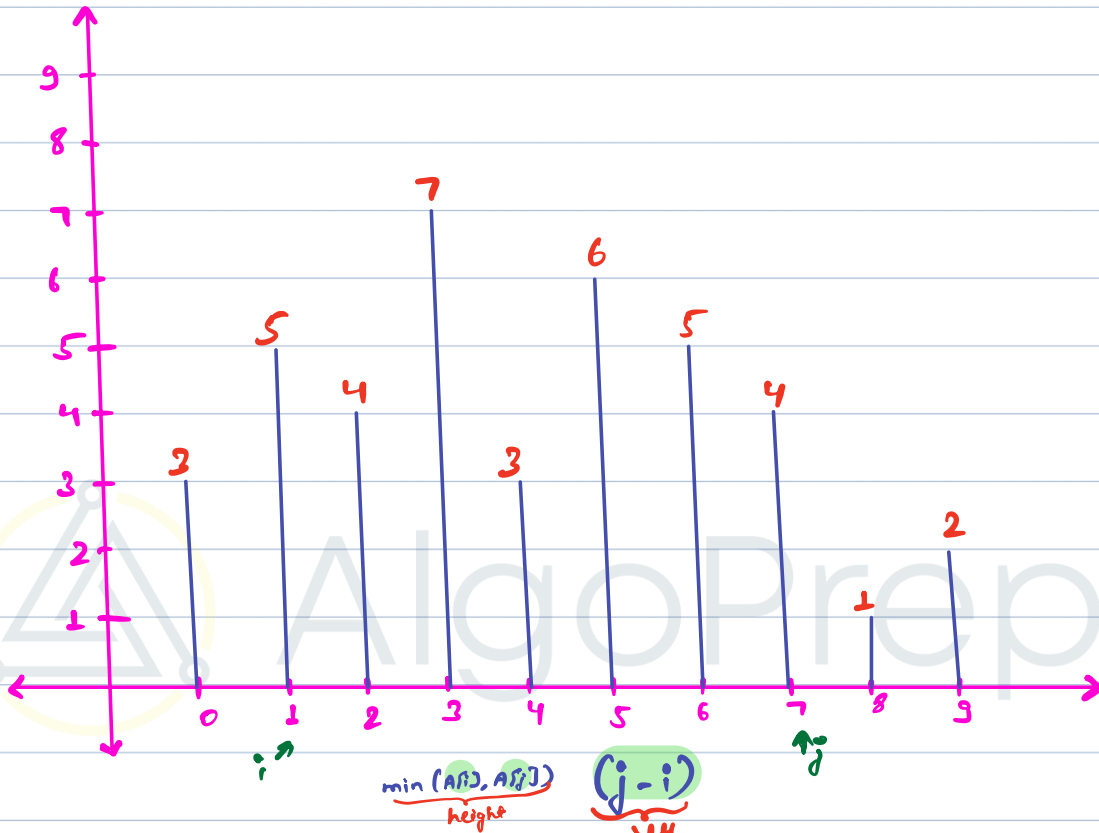
T.C: $O(N^2)$

S.C: $O(1)$

$$\text{* Amount of water } (i, j) = \underbrace{(j-i)}_{\text{width}} \times \underbrace{\min(arr[i], arr[j])}_{\text{height}}$$



$A[]: \{3, 5, 4, 7, 3, 6, 5, 4, 1, 2\}$



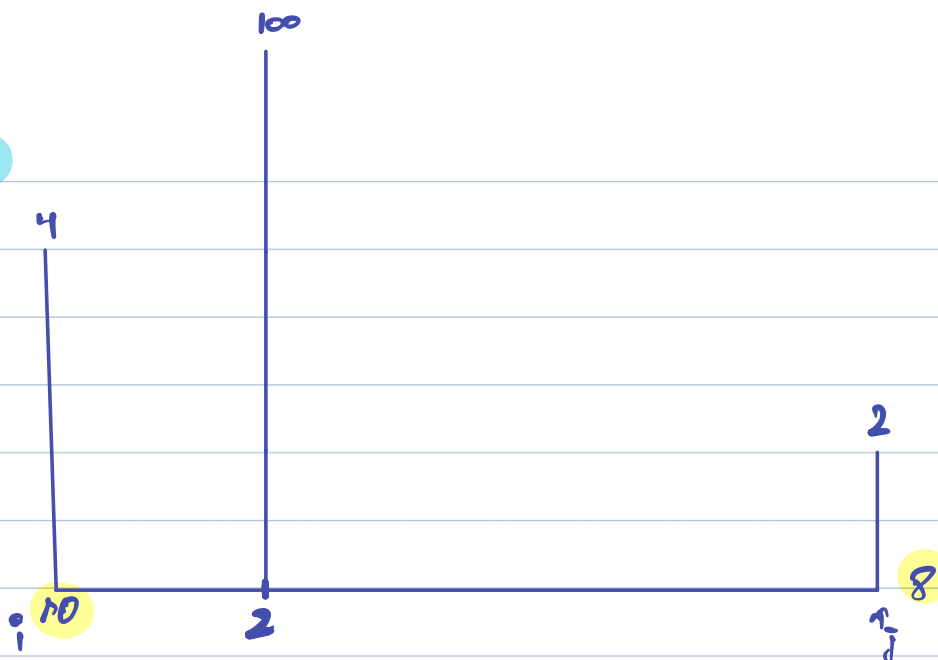
$A[i]$	$A[j]$	height	width	water	
3	2	2	9	18	$\rightarrow j--$
3	1	1	8	8	$\rightarrow j--$
3	4	3	7	21	$\rightarrow i++$
5	4	4	6	24	$\rightarrow j--$

$O(n^2) \rightarrow O(n)$
100 10

$(i < j)$



//why



$$(j-i)=8 \quad (j-i) \downarrow$$

$$(0,8) \quad (2,8)$$

$(0,8) \rightarrow \infty$

$(1,8)$

$(2,8)$

$(3,8)$

$(4,8)$

$(5,8)$

$(6,8)$

$(7,8)$

Case ①: $A[2] < A[0]$

↳ overall smaller

Case ②: $A[2] == A[0]$

$$(j-i)(0,8) > (j-i)(2,8) +$$

height are same

↳ overall smaller

Case ③: $A[2] > A[0]$

width $\rightarrow (0,8) > (2,8) \downarrow +$

Height $\rightarrow$ height will remain same. $\checkmark$

$$\min(A[i], A[j])$$

height



//Pseudo code

T.C: $O(N)$

S.C: $O(1)$

```
int maxWater (int arr[N]) {
    int i=0;
    int j=N-1;
    int ans = -1;

    while (i < j) {
        int w = (j-i) * min(arr[i], arr[j]);

        ans = max(ans, w);

        if (arr[i] <= arr[j]) {
            i++;
        }
        else {
            j--;
        }
    }

    return ans;
}
```



Q) Sort 0, 1 and 2

↳ Given an $arr[N]$ containing only 0, 1 and 2. Sort the array in ascending order.

Ex: $arr[10]$: { 0 1 2 1 2 1 0 0 1 0 }
↳ 0 0 0 0 1 1 1 1 2 2

//idea 1

↳ Sort the entire array.

T.C: $O(N \log N)$

S.C: $O(1)$

//idea 2

↳ Count sort

T.C: $O(N + \frac{N^3}{n^2}) = O(N)$

S.C: $O(1)$

$(0, i-1) \rightarrow \underline{0 \ 0 \ 0}$
 $(k+1, n-1) \rightarrow \underline{2 \ 2 \ 2}$
 $(i, k) \rightarrow \underline{1 \ 1 \ 1}$

||idea3

$(0, i-1) \rightarrow \underline{0 \ 0 \ 0}$
 $(i, j-1) \rightarrow \underline{1 \ 1 \ 1}$
 $(j, k) \rightarrow \text{to be solved}$
 $(k+1, n-1) \rightarrow \underline{2 \ 2 \ 2 \dots}$



$arr[10]: \{ 0 \ 1 \ 0 \ 1 \ 2 \ 1 \ 0 \ 0 \ 1 \ 2 \}$

$\rightarrow arr[j] == 0 \rightarrow \text{swap}(i, j)$
 $\hookrightarrow arr[j] == 1 \rightarrow j++$

$\hookrightarrow arr[j] == 2 \rightarrow \text{swap}(j, k)$
 $\hookrightarrow k--$

$arr[10]: \{ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 2 \ 2 \}$

$(0, i-1) \rightarrow \underline{0 \ 0 \ 0}$
 $(i, j-1) \rightarrow \underline{1 \ 1 \ 1}$
 $(j, k) \rightarrow \text{to be solved}$
 $(k+1, n-1) \rightarrow \underline{2 \ 2 \ 2 \dots}$

→ $arr[j] == 0 \rightarrow \text{swap}(i, j)$
↳ $i++$
↳ $j++$



// Pseudo code

↳ $arr[j] == 1 \rightarrow j++$

↳ $arr[j] == 2 \rightarrow \text{swap}(j, k)$
↳ $k--$

```
void sort012 (int arr[]) {
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    int k = n-1;
```

```
    while (j <= k)
```

```
    { if (arr[j] == 0) {  
        swap(i, j);  
        i++; j++;  
    }
```

```
    else if (arr[j] == 1) {  
        j++;  
    }
```

```
    else {  
        swap(j, k);  
        k--;  
    }
```

```
}
```

T.C: $O(n)$

S.C: $O(1)$

↳ Dutch national flag algo.